

OOP Principles: Encapsulation, Inheritance, Polymorphism, Abstraction

For a **3-year Java developer interview**, don't answer these as textbook definitions only. The interviewer will usually follow up with:

“Give me an example from your project.”

So learn the **concept** → **Java implementation** → **project example** → **why it matters**.

1. Encapsulation

Definition

Encapsulation means bundling data and the methods that operate on that data together, while restricting direct access to the internal state.

In Java, this is commonly achieved using:

- `private` fields
- public/protected methods
- getters/setters where appropriate

Example

```
public class Payment {  
  
    private BigDecimal amount;  
    private String status;  
  
    public BigDecimal getAmount() {  
        return amount;  
    }  
  
    public void updateStatus(String status) {  
        if (status == null || status.isBlank()) {  
            throw new IllegalArgumentException("Status cannot be empty");  
        }  
  
        this.status = status;  
    }  
}
```

Here:

```
Payment  
├─ amount  
├─ status  
└─ methods  
    └─ updateStatus()
```

The caller cannot directly do:

```
payment.status = "SUCCESS";
```

because `status` is private.

Instead:

```
payment.updateStatus("SUCCESS");
```

The class controls **how its internal state can change**.

Why is encapsulation useful?

Suppose payment status should only move:

```
INITIATED → PROCESSING → SUCCESS
```

You don't want arbitrary code to execute:

```
payment.status = "SUCCESS";
```

because that might bypass business rules.

Instead:

```
payment.markSuccessful();
```

can enforce the rules.

Interview answer

“Encapsulation means hiding an object's internal state and controlling access to it through well-defined methods. In Java, we commonly achieve this with private fields and public methods. It helps protect object state and keeps business rules within the class.”

2. Inheritance

Definition

Inheritance allows one class to acquire properties and behavior from another class.

Java uses:

```
extends
```

Example:

```
public class PaymentException extends RuntimeException {  
}
```

Here:

```
RuntimeException  
↑
```

```
PaymentException
```

`PaymentException` inherits behavior from `RuntimeException`.

Another example:

```
public class Payment {  
  
    protected String transactionId;  
  
    public void process() {  
        System.out.println("Processing payment");  
    }  
}
```

```
public class CashPayment extends Payment {  
  
    public void validateCash() {  
        System.out.println("Validating cash");  
    }  
}
```

`CashPayment` inherits from `Payment`.

Types of inheritance in Java

Java supports:

Single inheritance

```
A  
↑  
B
```

```
class B extends A
```

Multilevel inheritance

```
A  
↑  
B  
↑  
C
```

Java does **not** support multiple inheritance of classes:

```
class C extends A, B // invalid
```

But Java supports multiple inheritance of **type through interfaces**:

```
class PaymentService implements OnlinePayment, Auditable
```

Interview trap

Is inheritance always good?

No.

Don't say:

“Inheritance improves code reuse, therefore we should use it whenever possible.”

Inheritance creates tight coupling between parent and child.

Often **composition is preferred over inheritance**.

For example:

```
class PaymentService {  
  
    private PaymentValidator validator;  
}
```

rather than creating a deep inheritance hierarchy.

Interview answer

“Inheritance represents an IS-A relationship and allows a subclass to reuse or specialize parent behavior. In Java, classes support single inheritance, while multiple interfaces can be implemented. I use inheritance when the relationship is genuinely hierarchical; otherwise composition is often better.”

3. Polymorphism

Polymorphism means one interface/reference can represent different implementations, with the appropriate behavior selected at runtime or compile time depending on the type of polymorphism.

There are two important forms in Java.

A. Compile-time polymorphism

Usually achieved through **method overloading**.

```
public class PaymentService {  
  
    public void processPayment(String transactionId) {  
        System.out.println("Processing transaction");  
    }  
  
    public void processPayment(String transactionId, BigDecimal amount) {  
        System.out.println("Processing transaction with amount");  
    }  
}
```

Same method name:

```
processPayment()
```

but different parameters.

The compiler determines which method to call.

B. Runtime polymorphism

Achieved through **method overriding**.

This is more important for backend interviews.

```
interface PaymentProcessor {  
  
    void process();  
}
```

Implementation 1:

```
class CardPaymentProcessor implements PaymentProcessor {  
  
    @Override  
    public void process() {  
        System.out.println("Processing card payment");  
    }  
}
```

Implementation 2:

```
class CashPaymentProcessor implements PaymentProcessor {  
  
    @Override  
    public void process() {  
        System.out.println("Processing cash payment");  
    }  
}
```

Now:

```
PaymentProcessor processor =  
    new CardPaymentProcessor();  
  
processor.process();
```

prints:

```
Processing card payment
```

Change implementation:

```
PaymentProcessor processor =  
    new CashPaymentProcessor();  
  
processor.process();
```

prints:

```
Processing cash payment
```

The reference is:

```
PaymentProcessor
```

but the actual object determines the behavior.

4. Why polymorphism matters in your payment project

Imagine BPS supports:

```
CARD  
CASH  
UPI  
BANK_TRANSFER
```

Instead of:

```
if (type.equals("CARD")) {  
    ...  
} else if (type.equals("CASH")) {  
    ...  
} else if (type.equals("UPI")) {  
    ...  
}
```

you could define:

```
public interface PaymentProcessor {  
    PaymentResponse process(PaymentRequest request);  
}
```

Then:

```
CardPaymentProcessor  
CashPaymentProcessor  
UpiPaymentProcessor  
BankTransferProcessor
```

all implement:

```
PaymentProcessor
```

The service can work with the interface:

```
public PaymentResponse process(  
    PaymentProcessor processor,  
    PaymentRequest request) {  
  
    return processor.process(request);  
}
```

This gives you flexibility to add new payment types without changing the core processing logic excessively.

Interview answer

“Polymorphism allows a common interface or parent type to represent different implementations. In a payment system, different payment processors can implement the same `PaymentProcessor` interface, and the appropriate implementation can be selected at runtime.”

5. Abstraction

Definition

Abstraction means exposing essential behavior while hiding implementation details.

In Java, abstraction is commonly achieved through:

- interfaces
- abstract classes

Interface example

```
public interface PaymentProcessor {  
  
    PaymentResponse process(PaymentRequest request);  
}
```

The interface says:

A payment processor must provide `process()`.

It doesn't say **how** it should process the payment.

Implementation:

```
public class CashPaymentProcessor  
    implements PaymentProcessor {  
  
    @Override  
    public PaymentResponse process(PaymentRequest request) {  
        // cash-specific implementation  
        return new PaymentResponse();  
    }  
}
```

The caller doesn't need to know the internal implementation.

6. Abstract class

```
public abstract class PaymentProcessor {  
  
    public abstract void process();  
  
    public void validate() {  
        System.out.println("Common validation");  
    }  
}
```

```
}  
}
```

Child:

```
public class CashPaymentProcessor  
    extends PaymentProcessor {  
  
    @Override  
    public void process() {  
        System.out.println("Process cash");  
    }  
}
```

An abstract class can contain:

- abstract methods
- concrete methods
- instance variables
- constructors

An interface can also have default/static methods in modern Java, but conceptually an interface is primarily used to define a contract.

7. Abstraction vs Encapsulation

This is a **very common interview question**.

Encapsulation

Focuses on:

▮ **Hiding/protecting internal state and controlling access.**

Example:

```
private BigDecimal amount;
```

Abstraction

Focuses on:

▮ **Hiding implementation complexity and exposing essential behavior.**

Example:

```
paymentProcessor.process(request);
```

The caller doesn't care how the payment is processed.

Easy way to remember

```
Encapsulation  
→ "How do I protect the internals?"  
  
Abstraction  
→ "What do I expose and what implementation do I hide?"
```

8. All Four Together

Use a payment system example.

```
interface PaymentProcessor {  
    PaymentResponse process(PaymentRequest request);  
}
```

Abstraction

The interface defines:

```
process()
```

without exposing implementation.

Polymorphism

Different classes implement it:

```
CardPaymentProcessor  
CashPaymentProcessor  
UPIPaymentProcessor
```

and the application can use:

```
PaymentProcessor processor;
```

Encapsulation

Each processor hides its internal implementation:

```
private GatewayClient gatewayClient;  
private PaymentValidator validator;
```

Inheritance

Potentially, common behavior can be placed in a base class:

```
abstract class BasePaymentProcessor  
    implements PaymentProcessor {  
  
    protected void validateRequest(...) {  
        ...  
    }  
}
```

Then:

```
BasePaymentProcessor  
  ↑  
├─ CardPaymentProcessor  
├─ CashPaymentProcessor  
└─ UpiPaymentProcessor
```

Though in a real system, I'd avoid inheritance unless the common behavior genuinely belongs in an IS-A hierarchy.

9. Very Important: Interface vs Abstract Class

Expect this question.

Interface	Abstract Class
Defines a contract	Can define contract + shared implementation
A class can implement multiple interfaces	A class can extend only one class
No normal instance state in the traditional sense	Can have instance variables
Useful for loose coupling	Useful when subclasses genuinely share behavior/state
implements	extends

Example:

```
interface PaymentProcessor {  
    void process();  
}
```

vs:

```
abstract class BasePaymentProcessor {  
  
    protected void validate() {  
        ...  
    }  
  
    abstract void process();  
}
```

10. Interview Questions You Should Prepare

Basic

1. What are the four pillars of OOP?
2. Explain encapsulation.
3. Explain inheritance.
4. Explain polymorphism.
5. Explain abstraction.
6. Give a real-world example of each.

Java-specific

7. How is encapsulation implemented in Java?
8. What is method overloading?
9. What is method overriding?
10. Overloading vs overriding?
11. What is runtime polymorphism?
12. Does Java support multiple inheritance?
13. How does Java achieve multiple inheritance through interfaces?
14. Interface vs abstract class?
15. Composition vs inheritance?

Cross-questions

16. Why is composition often preferred over inheritance?
17. Is encapsulation achieved only through getters/setters?
18. Can private methods be overridden?
19. Can static methods be overridden?
20. Can constructors be inherited?
21. Can an abstract class have a constructor?
22. Can an interface have method implementations?
23. What happens when parent and child have the same method?
24. Explain upcasting and downcasting.
25. What is dynamic method dispatch?

11. Especially prepare these 5

For a **3-year Java interview**, I'd make sure you can answer these without hesitation:

1. Overloading vs Overriding

```
Overloading
→ Same method name
→ Different parameters
→ Compile time

Overriding
→ Same method signature
→ Child provides implementation
→ Runtime
```

2. Abstraction vs Encapsulation

```
Abstraction
→ Hide implementation complexity

Encapsulation
→ Protect internal state
```

3. Interface vs Abstract Class

Know when and why you'd select either.

4. Composition vs Inheritance

```
Inheritance
```

→ IS-A

Composition

→ HAS-A

Example:

CashPaymentProcessor IS-A PaymentProcessor

PaymentService HAS-A PaymentValidator

5. Runtime Polymorphism

Be able to explain this code:

```
PaymentProcessor processor =  
    new CashPaymentProcessor();  
  
processor.process();
```

and explain **why the CashPaymentProcessor implementation executes.**

Interview-ready answer

If the interviewer asks:

“**Explain the four pillars of OOP with examples.**”

A strong 3-year answer would be:

“The four main OOP principles are encapsulation, inheritance, polymorphism and abstraction. Encapsulation protects an object’s internal state and controls how it can be modified. Inheritance allows a class to reuse or specialize behavior from a parent class. Polymorphism allows the same interface or parent reference to work with different implementations, with runtime polymorphism being achieved through method overriding. Abstraction exposes the essential behavior while hiding implementation details, commonly using interfaces or abstract classes.

For example, in a payment system, I could define a `PaymentProcessor` interface for abstraction, have different processors such as `CashPaymentProcessor` and `CardPaymentProcessor` for polymorphism, encapsulate their internal clients and validation logic, and use inheritance only where there is genuinely shared behavior.”

That answer gives the interviewer several areas to drill into—which is exactly what you want when you can explain your project properly.